

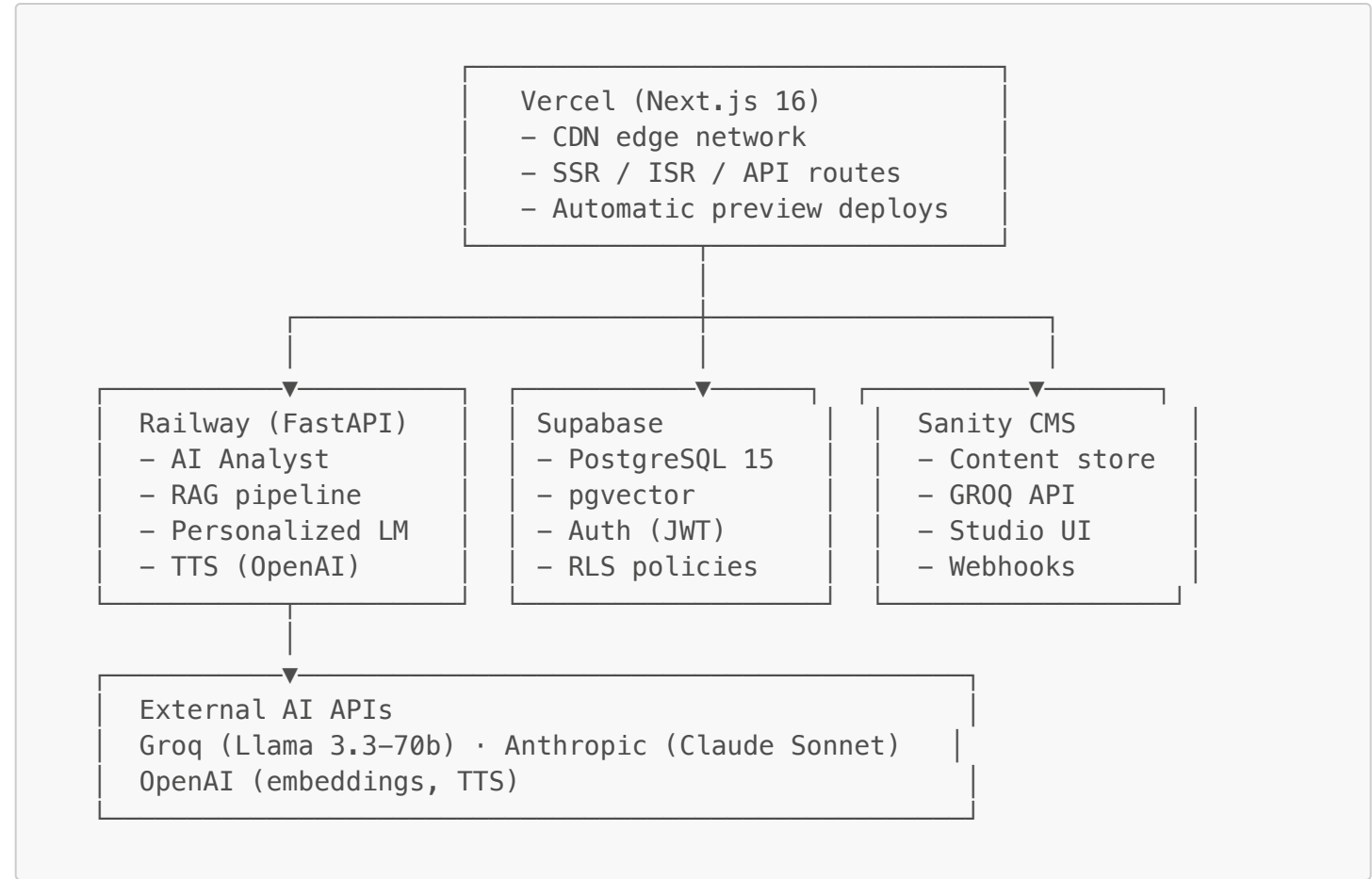
Operations Guide — Vermont Health Platform (HTR)

Audience: DevOps engineers, SREs, platform administrators. **Version:** 4.2.0 **Hosting:** Vercel (frontend) · Railway (backend) · Supabase (database) · Sanity (CMS)

Table of Contents

- 1. [Infrastructure Overview](#)
- 2. [Deployment — Frontend \(Vercel\)](#)
- 3. [Deployment — Backend \(Railway\)](#)
- 4. [Environment Management](#)
- 5. [Webhook Configuration](#)
- 6. [Monitoring & Alerting](#)
- 7. [Log Management](#)
- 8. [Database Operations](#)
- 9. [Incident Response](#)
- 10. [Runbooks](#)
- 11. [Security Operations](#)
- 12. [Cost Management](#)

1. Infrastructure Overview



Service Responsibilities

Service	Responsibility	SLA Target
Vercel	Frontend serving, API proxying, CDN	99.9%
Railway	AI inference, RAG, TTS	99.5%
Supabase	Database, auth, vector store	99.9%
Sanity	Content management, GROQ API	99.9%
Stripe	Payment processing	99.99%

Service	Responsibility	SLA Target
Groq	LLM inference (subscriber tier)	99.5%
Anthropic	LLM inference (professional tier)	99.5%
OpenAI	Embeddings, TTS	99.9%

2. Deployment — Frontend (Vercel)

Production Deployment

Every push to the **main** branch triggers an automatic Vercel production deployment. The deployment pipeline:

1. Vercel detects push to **main**
2. Runs **npm run build** in **frontend/**
3. Runs type checking and linting
4. Deploys to production edge network
5. Previous deployment stays live until new one is healthy

Build command: **npm run build** **Output directory:** **.next** **Root directory:** **frontend/** **Node version:** 20.x

Preview Deployments

Every pull request gets an automatic preview URL (e.g., **htr-pr-123.vercel.app**). Preview deployments use the same environment variables as production except **NEXT_PUBLIC_VERCEL_ENV=preview**.

Manual Deployment

```
# Install Vercel CLI
npm install -g vercel

# Deploy to production
vercel --prod

# Deploy preview
vercel
```

Rollback

In the Vercel dashboard → Deployments → click the previous deployment → Promote to Production. Rollback takes ~30 seconds.

Environment Variables

Set in Vercel dashboard → Project → Settings → Environment Variables. Variables prefixed with **NEXT_PUBLIC_** are bundled into the client-side JavaScript. Server-only variables (like **SUPABASE_SERVICE_ROLE_KEY**) must NOT have the **NEXT_PUBLIC_** prefix.

Critical variables (missing any of these will break core functionality):

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
NEXT_PUBLIC_SANITY_PROJECT_ID
NEXT_PUBLIC_SANITY_DATASET
SANITY_API_TOKEN
PYTHON_BACKEND_URL
INGEST_SECRET
```

3. Deployment — Backend (Railway)

Production Deployment

Every push to **main** triggers an automatic Railway deployment. The start command is in **backend/Procfile**:

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

Railway auto-sets **PORT**. Do not hardcode port 8000 in production.

Startup Sequence

On startup, the FastAPI app:

- 1. Initializes Sentry (if **SENTRY_DSN** set)
- 2. Calls **init_global_settings()** — configures LlamaIndex with Groq LLM and OpenAI embeddings
- 3. Warms up FlashRank re-ranker in a background thread
- 4. Attempts to load the existing vector index from pgvector
- 5. If no index exists, runs **build_index()** — fetches Sanity content, chunks, embeds, stores in pgvector

First startup after a new environment (no existing index) takes **5–15 minutes** depending on Sanity content volume. Subsequent restarts load the existing index in under 30 seconds.

Health Check

Railway monitors the **/health** endpoint. A non-200 response triggers a restart.

```
curl https://your-app.railway.app/health
```

Expected response:

```
{
  "status": "ok",
  "version": "4.2.0",
  "index_ready": true,
  "model_subscriber": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
  "vector_store": "pgvector",
  "auth_enabled": true,
  "reranker": "flashrank",
  "retrieval": "hybrid_bm25_vector_rrf",
  "chunking": "sentence_window_3"
}
```

If **index_ready** is **false**, the AI Analyst will return errors. Check Railway logs for build failures.

Manual Restart

Railway dashboard → your service → Restart. Or via CLI:

```
railway restart
```

Scaling

Railway scales vertically. If AI response latency increases under load, upgrade to a higher-memory instance. The vector store and LLM calls are the primary bottlenecks — horizontal scaling requires session affinity or shared

state, which the current in-process index design does not support without modification.

4. Environment Management

Environment Tiers

Tier	Frontend	Backend	Database
Production	Vercel production	Railway production	Supabase production project
Preview	Vercel preview URLs	Same as production	Same Supabase project (be careful)
Local	localhost:3000	localhost:8000	Same Supabase project or local Docker

Warning: Preview deployments share the production Supabase database by default. Test destructive operations locally with a separate Supabase project.

Secret Rotation

When rotating a secret:

1. Add the new value alongside the old in Vercel/Railway as a staging variable
2. Deploy and verify the new value works
3. Remove the old value
4. For `SUPABASE_JWT_SECRET`: this must match exactly between Supabase and the backend — rotate both simultaneously and trigger a backend restart

.env Files

File	Purpose	Committed?
frontend/.env.local	Local overrides	No (gitignored)
frontend/.env.production.example	Template for production vars	Yes
backend/.env	Local backend secrets	No (gitignored)
backend/.env.example	Template	Yes

5. Webhook Configuration

Sanity → Platform (Content Sync)

When content is published in Sanity, a webhook triggers the AI re-indexing pipeline.

Webhook target: `https://your-domain.com/api/webhooks/sanity` **Method:** POST **Trigger events:** Create, Update, Delete **Dataset:** production

Configuration in Sanity Manage → your project → API → Webhooks → Add:

```
URL: https://your-domain.com/api/webhooks/sanity
Secret: [SANITY_WEBHOOK_SECRET value]
HTTP method: POST
Trigger on: Create, Update, Delete
Include drafts: No
Filter: (none – trigger on all types)
Projection: { _id, _type }
```

The webhook handler in `frontend/app/api/webhooks/sanity/route.ts` verifies the signature and proxies a `POST /api/ingest` request to the Railway backend.

Stripe → Platform (Payment Events)

Webhook target: `https://your-domain.com/api/stripe/webhook` **Required events:**

Event	Action
<code>checkout.session.completed</code>	Create subscription record, grant role
<code>customer.subscription.updated</code>	Update plan/status
<code>customer.subscription.deleted</code>	Revoke subscriber role
<code>invoice.payment_failed</code>	Update status to <code>past_due</code>

Configuration in Stripe Dashboard → Developers → Webhooks → Add endpoint. Copy the webhook signing secret to `STRIPE_WEBHOOK_SECRET`.

Verifying Webhook Delivery

Check delivery status in:

- Sanity: Manage → API → Webhooks → your webhook → Activity log
- Stripe: Dashboard → Developers → Webhooks → your endpoint → Recent deliveries

6. Monitoring & Alerting

Sentry — Error Tracking

Both frontend and backend report to Sentry. Errors include full stack traces, user context (anonymized), and request metadata.

Frontend: `@sentry/nextjs` captures unhandled exceptions, API route errors, and client-side crashes.

Backend: `sentry-sdk` with `FastApiIntegration` and `StarletteIntegration` captures endpoint errors and slow transactions.

Traces sample rate: 10% (`traces_sample_rate=0.1`). Increase for debugging performance issues, decrease to reduce Sentry quota usage.

Configure alerts in Sentry → Alerts → Create Alert Rule:

- Issue alert: any new error → Slack notification
- Performance alert: P95 response time > 3s → Slack notification

Vercel Analytics

Core Web Vitals (LCP, INP, CLS) are reported via `web-vitals` integration. View in Vercel dashboard → Analytics.

Target thresholds:

Metric	Target	Poor
LCP	< 2.5s	> 4.0s
INP	< 200ms	> 500ms
CLS	< 0.1	> 0.25

Railway Metrics

Railway provides CPU, memory, and request count graphs in the dashboard. Set up Railway alerting for:

- Memory usage > 80% — consider upgrading instance
- Restart count > 2 in 1 hour — investigate crash loop

Supabase Monitoring

Supabase dashboard → Reports shows:

- Query performance: slowest queries by execution time
- API requests: request volume and error rates
- Auth: signup/login counts, error rate

Alert on: authentication error rate > 1% or database CPU > 70%.

7. Log Management

Frontend Logs (Vercel)

Vercel captures all `console.log`, `console.error`, and unhandled exceptions from Next.js API routes. View in Vercel dashboard → Logs.

Runtime logs are retained for 1 day on Hobby/Pro plans. For longer retention, configure a log drain to Datadog, Logtail, or similar.

Backend Logs (Railway)

All Python `logging` output appears in Railway → your service → Logs. The backend uses structured log format:

```
2026-03-15 10:23:11 INFO htr-brain: Chat complete - 842ms
2026-03-15 10:23:09 INFO htr-brain: Hybrid search returned 18 nodes
2026-03-15 10:23:09 WARNING htr-brain: Hybrid search failed: ... - falling back
```

Log levels: `INFO` for normal operation, `WARNING` for degraded fallbacks, `ERROR` for unhandled exceptions.

Railway retains logs for 7 days. Export to external storage for compliance or longer retention.

Useful Log Queries

```
# Backend - errors only
railway logs --filter "ERROR"

# Backend - slow responses (>2s)
railway logs | grep "Chat complete" | awk -F'-' '{if ($2+0 > 2000) print}'

# Zero-result queries (content gaps)
railway logs | grep "zero_result"
```

8. Database Operations

Connection Information

```
Host:      db.[project-id].supabase.co
Port:      5432
Database:  postgres
User:      postgres
SSL:       required
```

Connection strings are available in Supabase → Project Settings → Database → Connection string.

Backend uses: direct connection (`SUPABASE_DB_URL`) for pgvector operations. **Frontend uses:** Supabase REST API via `@supabase/ssr` — routes through PgBouncer.

Running SQL

For one-off administrative queries, use the Supabase SQL Editor (Dashboard → SQL Editor). For migration scripts, run them in order from [supabase/migrations/](#).

Backup Strategy

- **Automated:** Supabase Pro provides daily backups with 7-day PITR
- **Manual exports:** Run `pg_dump` from the connection info above before any schema migration
- **Critical tables:** `user_roles`, `subscriptions`, `stripe_events` — back up before Stripe webhook changes

Index Maintenance

pgvector HNSW indexes do not require manual maintenance. Standard B-tree indexes on high-write tables (`rag_query_log`) may benefit from periodic `VACUUM ANALYZE`:

```
VACUUM ANALYZE public.rag_query_log;
VACUUM ANALYZE public.rag_documents;
```

Supabase runs autovacuum by default. Manual VACUUM is only needed after bulk operations (mass inserts or deletes).

Re-ingesting the Knowledge Base

When significant new content is published or after a schema change to `rag_documents`:

```
# Trigger via API
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer YOUR_INGEST_SECRET"

# Check status
curl https://your-backend.railway.app/api/ingest/status \
  -H "Authorization: Bearer YOUR_INGEST_SECRET"
```

Or use the "Trigger Ingest" button in the admin dashboard at [/admin/ingest](#).

Re-ingestion clears `rag_documents` and rebuilds from scratch. During re-ingestion (~5–15 minutes), the AI Analyst continues serving responses from the existing in-memory index. The new index is swapped in atomically on completion.

9. Incident Response

Severity Levels

Level	Definition	Response Time
P0	Full platform outage or data breach	Immediate
P1	Core feature unavailable (AI Analyst, login)	< 30 minutes
P2	Degraded feature (slow responses, one tool broken)	< 2 hours
P3	Minor UI issue or non-critical feature broken	Next business day

P0 — Full Platform Outage

1. Check Vercel status: [vercel.com/status](#)
2. Check Supabase status: [status.supabase.com](#)
3. Check Railway status: [railway.app/status](#)
4. If a vendor issue: post status update, wait for resolution

- 5. If own code: check recent deployments → rollback if needed
- 6. If database: check Supabase connection limits and query load

P1 — AI Analyst Down

- 1. Check Railway health: `curl https://your-backend.railway.app/health`
- 2. If unhealthy: check Railway logs for startup errors
- 3. Common causes:
 - `OPENAI_API_KEY` invalid → embeddings fail during index build
 - `SUPABASE_JWT_SECRET` wrong → all authenticated requests fail with 401
 - pgvector extension dropped → index build fails
- 4. Fix the root cause, trigger a Railway restart
- 5. Monitor `/health` until `index_ready: true`

P1 — Login Broken

- 1. Check Supabase Auth status in Supabase dashboard
- 2. Check for expired JWT secret rotation — verify `SUPABASE_JWT_SECRET` matches in backend
- 3. Check CORS headers if login works locally but not in production

P2 — Slow AI Responses

- 1. Check `rag_query_log.latency_ms` for recent queries
- 2. If embedding is slow: check OpenAI API status
- 3. If retrieval is slow: check Supabase query performance in Dashboard → Reports
- 4. If LLM generation is slow: check Groq/Anthropic API status pages
- 5. If FlashRank is downloading: wait for first-time model download to complete

10. Runbooks

Runbook: Grant Admin Role to User

```
-- In Supabase SQL Editor
-- Replace 'user-uuid-here' with the user's UUID from auth.users
INSERT INTO public.user_roles (user_id, role, granted_by)
VALUES ('user-uuid-here', 'admin', 'your-admin-uuid')
ON CONFLICT (user_id, role) DO NOTHING;
```

Verify: ask the user to log out and back in (role is read from DB on each session refresh).

Runbook: Grant Subscriber Role After Manual Payment

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('user-uuid-here', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;

UPDATE public.subscriptions
SET plan = 'subscriber', status = 'active'
WHERE user_id = 'user-uuid-here';
```

Runbook: Revoke a Role

```
DELETE FROM public.user_roles
WHERE user_id = 'user-uuid-here' AND role = 'subscriber';
```

Runbook: Force Re-index After Sanity Content Update

```
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer $INGEST_SECRET"

# Poll until completed
watch -n 10 curl -s https://your-backend.railway.app/api/ingest/status \
  -H "Authorization: Bearer $INGEST_SECRET"
```

Runbook: Clear Stuck Ingest Job

```
-- In Supabase SQL Editor
UPDATE public.ingest_jobs
SET status = 'failed', error_message = 'Manually cleared by operator'
WHERE status IN ('queued', 'running')
  AND started_at < NOW() - INTERVAL '30 minutes';
```

Then restart the Railway backend to trigger a fresh build.

Runbook: Rotate API Key for a User

```
-- Revoke existing key
UPDATE public.api_keys
SET revoked_at = NOW()
WHERE user_id = 'user-uuid-here'
  AND revoked_at IS NULL;
```

Then have the user generate a new key from [/account](#) → API Keys.

Runbook: Debug Zero-Result AI Responses

```
-- Find recent zero-result queries
SELECT query, created_at, latency_ms
FROM public.rag_query_log
WHERE was_zero_result = TRUE
ORDER BY created_at DESC
LIMIT 20;
```

Common fixes:

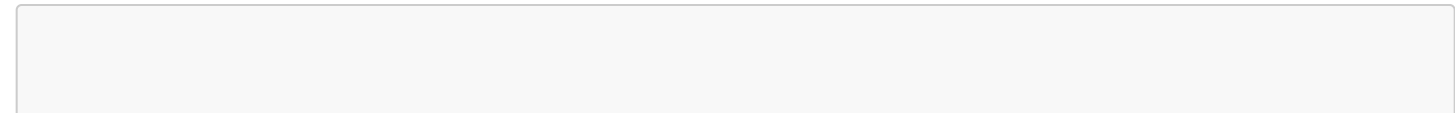
- Add relevant content to Sanity and trigger re-ingest
- Add a static document to [backend/data/](#) covering the topic
- Adjust the similarity threshold in [match_rag_documents](#) (lower threshold = more results, lower precision)

11. Security Operations

Access Control Review

Perform quarterly:

1. Audit [user_roles](#) for unexpected admin grants
2. Review [role_change_log](#) for unauthorized changes
3. Verify all API keys in [api_keys](#) are associated with active subscribers
4. Check [stripe_events](#) for any payment anomalies



```
-- Users with admin role
SELECT u.email, ur.granted_at, ur.granted_by
FROM public.user_roles ur
JOIN auth.users u ON u.id = ur.user_id
WHERE ur.role = 'admin'
ORDER BY ur.granted_at DESC;
```

API Key Security

API keys are stored as SHA-256 hashes. The plaintext key is shown once at creation. If a key is suspected compromised:

```
UPDATE public.api_keys
SET revoked_at = NOW()
WHERE key_hash = 'sha256-hash-of-compromised-key';
```

Rate Limiting

Rate limiting is enforced at two layers:

- 1. **Next.js middleware** — `/verify/` endpoint: 20 req/IP/60s (in-process **Map**, resets on cold start)
- 2. **FastAPI (slowapi)** — `/api/chat`: 30/min, `/api/suggest`: 60/min, `/api/ingest`: 5/hour

If a specific IP is abusing the platform, add it to Vercel's firewall rules (Vercel Pro/Enterprise) or block at the CDN level.

Secrets That Must Never Be Public

Secret	Consequence if leaked
SUPABASE_SERVICE_ROLE_KEY	Full database access, bypasses all RLS
SUPABASE_JWT_SECRET	Can forge valid JWT tokens for any user
STRIPE_SECRET_KEY	Can charge customers, issue refunds
STRIPE_WEBHOOK_SECRET	Can forge Stripe webhook events
INGEST_SECRET	Can trigger arbitrary re-indexing
ANTHROPIC_API_KEY	Unbounded API cost

If any of these are accidentally exposed (committed to git, logged, etc.): rotate immediately in the respective service dashboard, then update all environments.

12. Cost Management

Current Cost Drivers

Service	Cost Driver	Optimization
Groq	Token throughput (chat + suggestions)	Rate limiting, response length caps
OpenAI	Embedding API calls (ingest), TTS audio	Batch embeddings during ingest, cache TTS
Anthropic	Professional/Advisory tier chat	Only routed for highest tier
Railway	Compute hours	Auto-sleep on inactivity (dev), right-size prod
Supabase	Storage (pgvector), egress	Prune old <code>rag_query_log</code> rows quarterly
Vercel	Bandwidth, function invocations	ISR caching reduces function calls

Monitoring API Costs

- **Groq:** Dashboard → Usage — track tokens/day by model
- **OpenAI:** Platform → Usage — watch embeddings (ingest spikes) and TTS
- **Anthropic:** Console → Usage — professional tier usage volume

Set budget alerts in each API provider dashboard to notify at 80% of monthly budget.

Log Pruning

`rag_query_log` grows continuously. Prune records older than 90 days quarterly:

```
DELETE FROM public.rag_query_log
WHERE created_at < NOW() - INTERVAL '90 days';
```

Similarly prune old `ingest_jobs`:

```
DELETE FROM public.ingest_jobs
WHERE completed_at < NOW() - INTERVAL '30 days';
```